MOONVEIL

Talar árboles en dos fases y mochila de 5×5



Proyecto: Moonveil (Yellow Stone) · Unity 6000.6.0f1
Módulo: 2º DAM · Primer trimestre
Fecha: 26 de octubre de 2026 · **Segunda entrega**

Índice

1. Qué se ha hecho en esta entrega
2. Cómo se juega ahora
3. Archivos nuevos y modificados
4. Árboles en dos fases
5. El inventario (la mochila)
6. La ventana de la mochila
7. AyudaUI: no repetir código
8. Cambios pequeños en otros scripts
9. Gráficos nuevos
10. Cómo probarlo
11. Glosario de lo nuevo

1. Qué se ha hecho en esta entrega

Partimos del sistema de herramientas de la entrega anterior (azada, regadera, pico, hacha y semillas con su barra). Ahora se ha añadido:

- **Talar árboles con el hacha**, en **dos fases**:
    1. Al dar 4 golpes al árbol, se cae y queda **un tronco cortado a la mitad**.
    2. Al dar 2 golpes a ese tronco, **desaparece**.
- Un **inventario de 5×5** (25 huecos), al que llamamos "mochila", que guarda objetos. En cada hueco caben hasta 99 objetos iguales.
- Una **ventana** para ver la mochila, que se abre y se cierra con la tecla **I** (o Tab).
- Ahora romper cosas **da objetos**: madera, piedra y fibra. Las zanahorias cosechadas también van a la mochila.

> Igual que antes, **no se ha tocado la escena** `Granja.unity`. Todo se monta solo al darle a Play con el script `InstaladorHerramientas`, al que se le han añadido unas pocas líneas.

2. Cómo se juega ahora

Tecla	Acción
WASD / Flechas	Moverse
E	Cosechar zanahorias (ahora además van a la mochila)
1-5 / Rueda	Cambiar de herramienta
Espacio / Clic	Usar la herramienta (no funciona con la mochila abierta)
I o **Tab**	**NUEVO:** abrir y cerrar la mochila
Ratón encima de un hueco	**NUEVO:** el título de la mochila muestra el nombre y la cantidad del objeto

Qué da cada cosa al romperla

Qué se rompe	Herramienta	Golpes	Qué da
Árbol (pino, roble, arce, manzano)	Hacha	4	 3 Madera y se queda en tronco
Tronco cortado (lo que queda del árbol)	Hacha	2	 2 Madera
Tocón	Hacha	3	 2 Madera
Arbusto	Hacha	2	 2 Fibra
Hierba alta, flores, setas	Hacha	1	 1 Fibra
Roca	Pico	2	 1 Piedra
Roca grande	Pico	4	 3 Piedra
Zanahoria madura (tecla E)	—	—	 1 Zanahoria

3. Archivos nuevos y modificados

Scripts nuevos (carpeta `Assets/Scripts/Inventario/`)

Archivo	Para qué sirve
`Ranura.cs`	Un hueco de la mochila: nombre del objeto y cantidad.
`Inventario.cs`	La mochila: un array de 25 ranuras y los métodos `Guardar()` y `Contar()`.
`InterfazInventario.cs`	Dibuja la ventana de la mochila (cuadrícula 5×5) y la abre o cierra con I.
`AyudaUI.cs`	Métodos para crear Canvas, imágenes y textos por código. Los usan la barra y la mochila.

Scripts modificados

Archivo	Qué se ha cambiado
`ObjetoRompible.cs`	Puede tener una segunda fase (tronco) y da un objeto a la mochila al romperse.
`InstaladorHerramientas.cs`	Crea la mochila y su ventana, marca los árboles como rompibles y dice qué da cada cosa.
`InterfazHerramientas.cs`	Usa `AyudaUI` en vez de tener sus propios métodos de crear imágenes y textos.
`UsarHerramienta.cs`	Una condición más: no se usan herramientas con la mochila abierta.
`Cultivo.cs`	Una línea más: al cosechar, la zanahoria va a la mochila.

Gráficos nuevos (carpeta `Assets/Resources/Objetos/`)

Dibujo	Archivo	Qué es
	`tronco_cortado.png`	El tronco a la mitad que queda al talar un árbol (16×22).
	`madera.png`	Icono de la madera.
	`piedra.png`	Icono de la piedra.
	`fibra.png`	Icono de la fibra (un manojo de hierba atado).
	`zanahoria.png`	Icono de la zanahoria.

Truco importante: cada icono se llama igual que el objeto pero en minúsculas (objeto "*Madera*" → archivo `madera.png`). Así la mochila encuentra el dibujo de cualquier objeto con una sola línea: `Resources.Load<Sprite>("Objetos/" + nombre.ToLower())`. Para añadir un objeto nuevo basta con poner su PNG en esa carpeta.

4. Árboles en dos fases



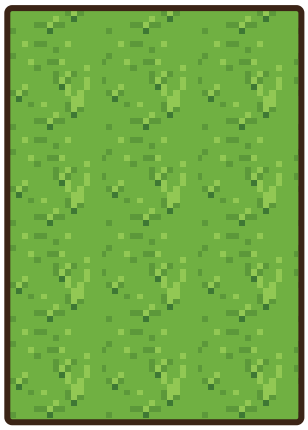
Fase 0: árbol
4 golpes

→
+3 Madera



Fase 1: tronco
2 golpes

→
+2 Madera



Fase 2: nada
se puede pasar

La idea

En vez de crear un script nuevo para los árboles, se ha **reutilizado** `ObjetoRompible`, que ya sabía contar golpes, temblar y desaparecer. Solo le faltaba una cosa: que, la primera vez que se queda sin golpes, en lugar de desaparecer **cambie su dibujo** por el del tronco y vuelva a tener golpes.

Para eso se añaden dos variables nuevas. Si `spriteTronco` está vacío (`null`), el objeto se comporta como antes (rocas, tocones...). Si tiene un sprite, es un árbol.

```
[Header("Solo para árboles")]
public Sprite spriteTronco;      // si tiene sprite, primero se convierte en tronco
public int golpesTronco = 2;     // golpes que aguanta el tronco
```

El método Golpear() nuevo

```
public void Golpear()
{
    if (roto) return;

    golpes--;

    // Todavía le quedan golpes: solo tiembla
    if (golpes > 0)
    {
        if (!temblando) StartCoroutine(Temblar());
        return;
    }

    // Se ha quedado sin golpes y es un árbol entero -> FASE 1: se queda en tronco
    if (spriteTronco != null)
    {
        GetComponent<SpriteRenderer>().sprite = spriteTronco;
        spriteTronco = null;      // ya no es un árbol: la próxima vez se rompe del todo
        golpes = golpesTronco;
        DarObjeto();
        if (!temblando) StartCoroutine(Temblar());
        return;
    }

    // FASE FINAL: se rompe y desaparece
    roto = true;
    DarObjeto();
    StartCoroutine(Romper());
}
```

Fíjate en el truco de la línea `spriteTronco = null;`: después de convertirse en tronco, borramos el sprite del tronco. Así, cuando el tronco se quede sin golpes, el `if (spriteTronco != null)` ya será falso y pasará a la fase final. **No hace falta una variable "fase"**: el propio sprite nos dice en qué fase está.

¿Por qué el tronco sale bien colocado?

Los sprites de los árboles tienen el **pivote abajo en el centro** (*Bottom Center*), es decir, la posición del objeto es la base del tronco. El tronco cortado se ha dibujado con el pivote en el mismo sitio, así que al cambiar el sprite queda justo en la base del árbol, sin tener que mover nada. El **collider** no cambia, porque en los árboles ya estaba solo en la base del tronco.

Dar objetos

```
void DarObjeto()
{
    if (objetoQueDa != "" && Inventario.instancia != null)
    {
        Inventario.instancia.Guardar(objetoQueDa, cantidadQueDa);
    }
}
```

Se comprueba que `Inventario.instancia` no sea `null` por si algún día se usa `ObjetoRompible` en una escena sin mochila: así no da error.

Cómo se marcan los árboles

En `InstaladorHerramientas`, igual que se hacía con las rocas, se buscan los objetos por su nombre:

```
else if (nombre.StartsWith("Pino") || nombre.StartsWith("Roble") || nombre.StartsWith("Arce") ||
nombre.StartsWith("Manzano"))
{
    ObjetoRompible arbol = HacerRompible(obj, TipoHerramienta.Hacha, 4, "Madera", 3);
    arbol.spriteTronco = Resources.Load<Sprite>("Objetos/tronco_cortado");
    arbol.golpesTronco = 2;
}
```

`HacerRompible` ahora recibe también qué objeto da y cuántos, y **devuelve** el componente creado (antes era `void`) para poder ponerle el sprite del tronco.

5. El inventario (la mochila)

Ranura.cs: un hueco

```
[System.Serializable]
public class Ranura
{
    public string nombre = "";
    public int cantidad = 0;

    public bool EstaVacia()
    {
        return nombre == "";
    }
}
```

Para guardar un objeto solo necesitamos su **nombre** (por ejemplo *"Madera"*) y **cuántos hay**. Un hueco vacío es el que tiene el nombre vacío.

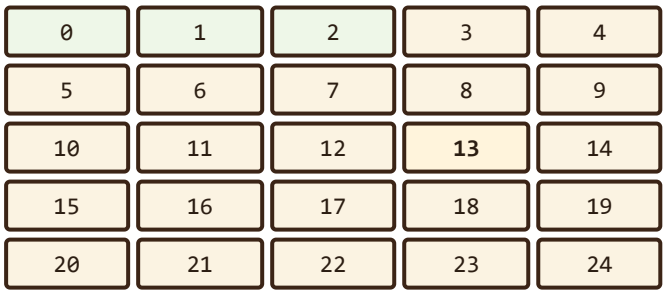
Inventario.cs: 25 huecos en un array

```
public const int FILAS = 5;
public const int COLUMNAS = 5;
public int maximoPorRanura = 99;

public Ranura[] ranuras = new Ranura[FILAS * COLUMNAS];   // 25 huecos
```

Aunque en pantalla se ve como una cuadrícula de 5×5, por dentro es un **array normal de 25 posiciones**. Para saber qué posición del array corresponde a una fila y columna se usa:

```
posición = fila * COLUMNAS + columna      // ejemplo: fila 2, columna 3  →  2 * 5 + 3 = 13
```



Las 25 posiciones del array colocadas en 5 filas. La 13 es la fila 2, columna 3 (se empieza a contar en 0).

Se usan **constantes** (`const`) para el tamaño: si algún día queremos una mochila de 6×6, solo hay que cambiar esos dos números y todo lo demás (array, ventana) se adapta.

En `Awake()` se rellena el array con ranuras vacías. Es importante: `new Ranura[25]` crea el array, pero sus 25 posiciones valen `null` hasta que les ponemos un objeto con `new Ranura()`.

```
void Awake()
{
    instancia = this;
    for (int i = 0; i < ranuras.Length; i++)
    {
        ranuras[i] = new Ranura();
    }
}
```

`instancia` es una variable **static**: guarda la mochila del jugador para que cualquier script pueda usarla con `Inventario.instancia.Guardar(...)`, igual que en el proyecto se usa `ControladorJuego.zanahorias`.

Guardar objetos

Cuando guardamos un objeto hay que decidir en qué hueco va. Se sigue la misma regla que en Stardew Valley:

1. Si ya hay un hueco con **ese mismo objeto** y no está lleno (menos de 99), va ahí (se "apila").
2. Si no, va al **primer hueco vacío**.
3. Si no hay ninguno, la mochila está llena.

```
int BuscarRanura(string nombre)
{
    for (int i = 0; i < ranuras.Length; i++)       // 1. mismo objeto y no lleno
    {
        if (ranuras[i].nombre == nombre && ranuras[i].cantidad < maximoPorRanura) return i;
    }
    for (int i = 0; i < ranuras.Length; i++)       // 2. primer hueco vacío
    {
        if (ranuras[i].EstaVacia()) return i;
    }
    return -1;                                      // 3. no hay sitio
}
```

Y `Guardar()` usa un `while` por si los objetos no caben todos en un hueco (por ejemplo, si tenemos 98 maderas y nos dan 3: entra 1 en ese hueco y 2 en otro):

```
public bool Guardar(string nombre, int cantidad)
{
    while (cantidad > 0)
    {
        int i = BuscarRanura(nombre);
        if (i == -1)
        {
            Debug.Log("La mochila está llena");
            return false;
        }

        ranuras[i].nombre = nombre;

        int cabe = maximoPorRanura - ranuras[i].cantidad;   // sitio libre en ese hueco
        int meter = Mathf.Min(cabe, cantidad);              // metemos lo que quepa
        ranuras[i].cantidad += meter;
        cantidad -= meter;                                  // lo que sobra, a la siguiente vuelta
    }
    return true;
}
```

Ejemplo	Qué pasa
Mochila vacía, `Guardar("Madera", 3)`	Hueco 0 = Madera ×3
Después, `Guardar("Piedra", 1)`	Hueco 1 = Piedra ×1
Después, `Guardar("Madera", 2)`	Hueco 0 = Madera ×5 (se apila con la que ya había)
Hueco 0 con Madera ×98, `Guardar("Madera", 3)`	Hueco 0 = ×99 y hueco 2 = Madera ×2

También hay un método `Contar(nombre)` que suma todas las cantidades de un objeto. Ahora mismo no se usa, pero servirá para cosas como "¿tengo 10 maderas para construir algo?".

6. La ventana de la mochila



Recreación de la mochila hecha con los sprites reales del proyecto. El ratón está encima de la madera, por eso ese hueco tiene el marco dorado y el título dice "Madera x5". Detrás, el juego se oscurece.

Cómo se construye

`InterfazInventario` crea todo por código en `Start()`, igual que la barra de herramientas:

1. Un **Canvas** con orden 20, para que quede por encima de la barra (que tiene 10).
2. Una imagen negra al 40 % de transparencia que ocupa toda la pantalla (oscurece el juego).
3. El panel de madera en el centro, con el título "*Mochila*".
4. Los 25 huecos con **dos bucles `for` anidados** (filas y columnas). Cada hueco tiene un icono y un texto con la cantidad.

```
for (int fila = 0; fila < Inventario.FILAS; fila++)
{
    for (int columna = 0; columna < Inventario.COLUMNAS; columna++)
    {
        int i = fila * Inventario.COLUMNAS + columna;   // posición en el array

        float x = margen + columna * (tamRanura + separacion);
        float y = -(margen + altoTitulo + fila * (tamRanura + separacion));   // hacia abajo es
negativo

        Image fondo = AyudaUI.CrearImagen("Hueco" + i, panel.transform, spriteRanura, 3);
        AyudaUI.Colocar(fondo.rectTransform, new Vector2(0f, 1f), new Vector2(x, y), new
Vector2(tamRanura, tamRanura));
        ...
    }
}
```

Los huecos se enganchan a la esquina de **arriba a la izquierda** del panel (ancla `(0, 1)`), por eso la *y* es negativa: cada fila está más abajo que la anterior.

Abrir y cerrar

```
if (teclado.iKey.wasPressedThisFrame || teclado.tabKey.wasPressedThisFrame)
{
    abierto = !abierto;          // si estaba abierto se cierra y al revés
    ventana.SetActive(abierto);  // SetActive enseña o esconde el objeto y todos sus hijos
}
```

`abierto` es `static` para que `UsarHerramienta` pueda consultarlo y no dé golpes mientras miramos la mochila:

```
if (pulsado && !usando && !InterfazInventario.abierto)
```

Pintar lo que hay en la mochila

Mientras está abierta, en cada frame se recorre el array y se pinta cada hueco:

```
for (int i = 0; i < inventario.ranuras.Length; i++)
{
    Ranura ranura = inventario.ranuras[i];

    if (ranura.EstaVacia())
    {
        iconos[i].enabled = false;     // sin dibujo
        cantidades[i].text = "";
    }
    else
    {
        iconos[i].enabled = true;
        iconos[i].sprite = Resources.Load<Sprite>("Objetos/" + ranura.nombre.ToLower());
        cantidades[i].text = ranura.cantidad.ToString();
    }
    ...
}
```

Si está cerrada, `Update()` termina enseguida con `if (!abierto) return;` para no gastar tiempo.

Ver el nombre con el ratón

El proyecto usa solo el Input System nuevo y el EventSystem de la escena no detecta el ratón sobre la interfaz, así que lo comprobamos a mano. `RectTransformUtility.RectangleContainsScreenPoint` nos dice si un punto de la pantalla (la posición del ratón) está dentro de un rectángulo de la interfaz:

```
bool RatonEncima(RectTransform rt)
{
    if (Mouse.current == null) return false;
    Vector2 posicionRaton = Mouse.current.position.ReadValue();
    return RectTransformUtility.RectangleContainsScreenPoint(rt, posicionRaton);
}
```

Si el ratón está encima de un hueco, se le pone el marco dorado (el mismo sprite que la casilla seleccionada de la barra) y el título cambia a "*Madera x5*".

7. AyudaUI: no repetir código

La barra de herramientas tenía sus propios métodos `CrearImagen()` y `CrearTexto()`. La mochila necesitaba exactamente los mismos. En vez de **copiar y pegar** (y tener que arreglar los fallos dos veces), se han movido a una clase `static` nueva, `AyudaUI`, que pueden usar las dos:

Método	Qué hace
`CrearCanvas(nombre, padre, orden)`	Crea un Canvas que se adapta al tamaño de la ventana.
`CrearImagen(nombre, padre, sprite, escalaPixel)`	Crea una imagen. Si `escalaPixel` > 0 usa 9-slice con los píxeles agrandados.
`CrearTexto(nombre, padre, texto, tamaño, color)`	Crea un texto en negrita con la fuente de Unity.
`Colocar(rt, ancla, posicion, tamaño)`	**Nuevo:** ancla, pivote, posición y tamaño en una sola línea (antes eran 5 líneas cada vez).

```
// Antes (en InterfazHerramientas):
Image fondo = CrearImagen("BarraHerramientas", canvas, spritePanel, 4);
// Ahora:
Image fondo = AyudaUI.CrearImagen("BarraHerramientas", canvas, spritePanel, 4);
```

Una clase `static` no se pone en ningún objeto ni se crea con `new`: sus métodos se llaman directamente con el nombre de la clase, como `Mathf.Lerp` o `Debug.Log`.

8. Cambios pequeños en otros scripts

En verde lo añadido y en rojo lo quitado.

InstaladorHerramientas.cs

```
+        // 4. La mochila (inventario de 5x5) y su ventana
+        Inventario mochila = jugador.AddComponent<Inventario>();
+        GameObject objMochila = new GameObject("InterfazInventario");
+        InterfazInventario interfazMochila = objMochila.AddComponent<InterfazInventario>();
+        interfazMochila.inventario = mochila;

...
-        if (nombre.StartsWith("Roca")) HacerRompible(obj, TipoHerramienta.Pico, 2);
+        if (nombre.StartsWith("Roca")) HacerRompible(obj, TipoHerramienta.Pico, 2, "Piedra", 1);
   // y así con el resto
...
-    static void HacerRompible(GameObject obj, TipoHerramienta herramienta, int golpes)
+    static ObjetoRompible HacerRompible(GameObject obj, TipoHerramienta herramienta, int golpes,
string objeto, int cantidad)

     {
         ...
+        rompible.objetoQueDa = objeto;
+        rompible.cantidadQueDa = cantidad;
+        return rompible;

     }
```

También se ha añadido "*I: mochila*" al texto de controles de la pantalla.

Cultivo.cs

```
             ControladorJuego.zanahorias++;
             ControladorJuego.semillas++;
+            if (Inventario.instancia != null) Inventario.instancia.Guardar("Zanahoria", 1); // NUEVO:
va a la mochila
```

UsarHerramienta.cs

```
-        if (pulsado && !usando)
+        if (pulsado && !usando && !InterfazInventario.abierto) // con la mochila abierta no se usan
herramientas
```

9. Gráficos nuevos



Tronco cortado, madera, piedra, fibra y zanahoria (ampliados 5 veces).

Se han hecho igual que los de la entrega anterior: dibujados como texto (una letra por color), convertidos a PNG con un pequeño script y con el contorno oscuro añadido automáticamente. Los colores del tronco se han sacado **del propio sprite del tocón y de los árboles** del mapa, para que parezca el mismo árbol cortado. Debajo lleva la misma sombra gris que los árboles.

Configuración: *Pixels Per Unit* 16, *Filter Mode* Point y sin compresión, como el resto. El tronco tiene el pivote abajo en el centro (como los árboles); los iconos, en el centro.

10. Cómo probarlo

1. Volver a Unity, esperar a que importe los PNG nuevos y compile, abrir la escena **Granja** y darle a **Play**.
2. Elegir el hacha (tecla 4) y ponerse delante de la base de un árbol.
    - Golpes 1-3: el árbol tiembla.
    - Golpe 4: el árbol se convierte en un tronco cortado.
    - Golpe 5: el tronco tiembla. Golpe 6: el tronco desaparece y ya se puede pasar por ahí.
3. Pulsar **I**: debe haber 5 de madera en el primer hueco. Pasar el ratón por encima: el título pone "Madera x5".
4. Con la mochila abierta, pulsar Espacio: no debe pasar nada. Pulsar I otra vez para cerrarla.
5. Romper una roca con el pico y cortar hierba con el hacha: aparecen Piedra y Fibra en huecos nuevos.
6. Cosechar una zanahoria madura con E: aparece Zanahoria en la mochila.

> Todos los scripts se han compilado con el compilador de C# que trae Unity 6000.6 y las librerías del proyecto, **sin errores ni avisos**. Como Unity estaba abierto, la prueba dentro del juego (la lista de arriba) queda pendiente de hacer en el editor.

11. Glosario de lo nuevo

Concepto	Explicación sencilla
Array	Lista de tamaño fijo. `new Ranura[25]` crea 25 posiciones, de la 0 a la 24, que empiezan valiendo `null`.
const	Valor que no cambia nunca mientras se ejecuta el programa (`FILAS = 5`).
Bucles anidados	Un `for` dentro de otro. Sirve para recorrer filas y columnas.
static (variable)	Una sola copia para toda la clase: `Inventario.instancia`, `InterfazInventario.abierto`.
static (clase)	Clase que solo tiene métodos de ayuda y no se crea con `new` (`AyudaUI`).
Awake	Se ejecuta antes que `Start`, en cuanto se crea el componente. Se usa para preparar los datos.
SetActive	Muestra u oculta un GameObject y todos sus hijos.
Mathf.Min	Devuelve el menor de dos números. Lo usamos para no meter más de lo que cabe.
ToLower	Pasa un texto a minúsculas ("Madera" → "madera").
null	"Nada". Un objeto que no existe. Siempre hay que comprobarlo antes de usarlo para evitar errores.
Pivote	El punto del sprite que coincide con la posición del objeto. En los árboles está abajo en el centro.
Reutilizar código	Aprovechar algo que ya funciona (`ObjetoRompible`, `AyudaUI`) en vez de escribirlo otra vez.